



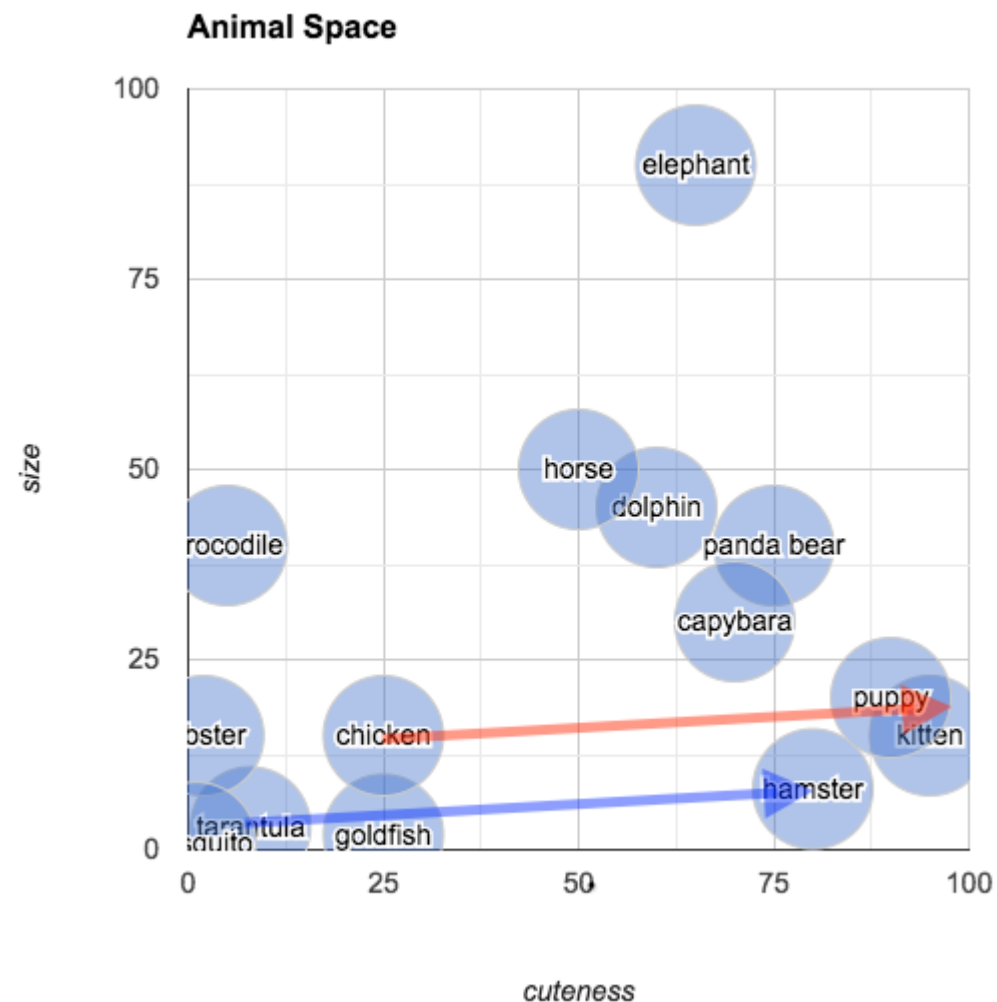
Topic : Semantic Processing

Agenda

1. Introduction to Word2Vec models and Setup with Gensim
2. Training Word2Vec Models and Analogies
3. Applying Vector Operations and Understanding Word Relationships
4. QnR

Vector- A quantity having direction as well as magnitude, especially as determining the position of one point in space relative to another.

	cuteness (0-100)	size (0-100)
kitten	95	15
hamster	80	8
tarantula	8	3
puppy	90	20
crocodile	5	40
dolphin	60	45
panda bear	75	40
lobster	2	15
capybara	70	30
elephant	65	90
mosquito	1	1
goldfish	25	2
horse	50	50
chicken	25	15



Each word is represented by a **1-hot vector** of size 10,000:

- All elements are zero except one position that corresponds to the word's index.
- E.g., for the word "Orange" (index 6257), the one-hot vector has 1 at position 6257, rest are 0.

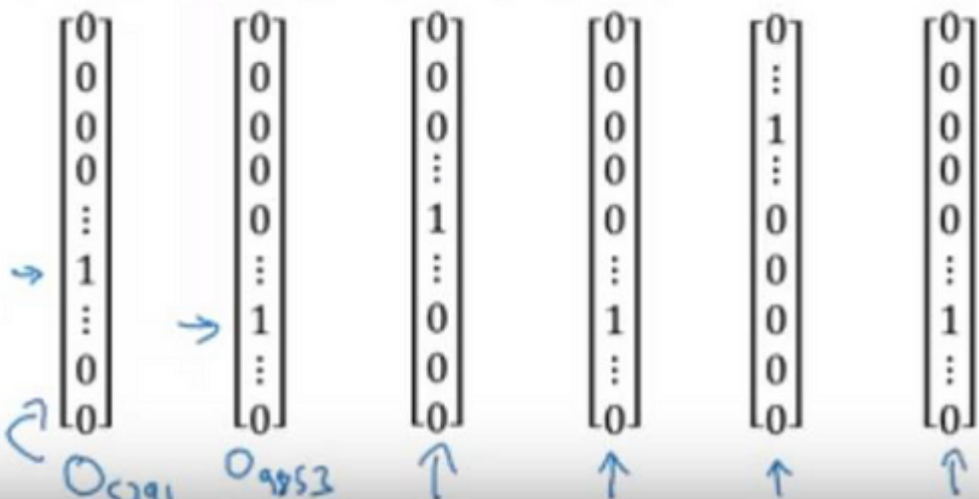
Word representation

$V = [a, aaron, \dots, zulu, <UNK>]$

$|V| = 10,000$

1-hot representation

Man (5391) Woman (9853) King (4914) Queen (7157) Apple (456) Orange (6257)



I want a glass of orange juice.
I want a glass of apple ____.

These vectors are **sparse**, **high-dimensional**, and **do not capture meaning or relationships** between words.

Instead of sparse one-hot vectors, we now represent each word with a **dense vector** of real numbers (size 300), capturing **semantic features** like **gender**, **royalty**, **age**, or **food**.

Featurized representation: word embedding

	Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	0.93	0.95	-0.01	0.00
Age	0.03	0.02	0.7	0.69	0.03	-0.02
Food	0.04	0.01	0.02	0.01	0.95	0.97
size	...	...				
cost						
align						
verb						

I want a glass of orange juice.

I want a glass of apple juice.

Andrew Ng

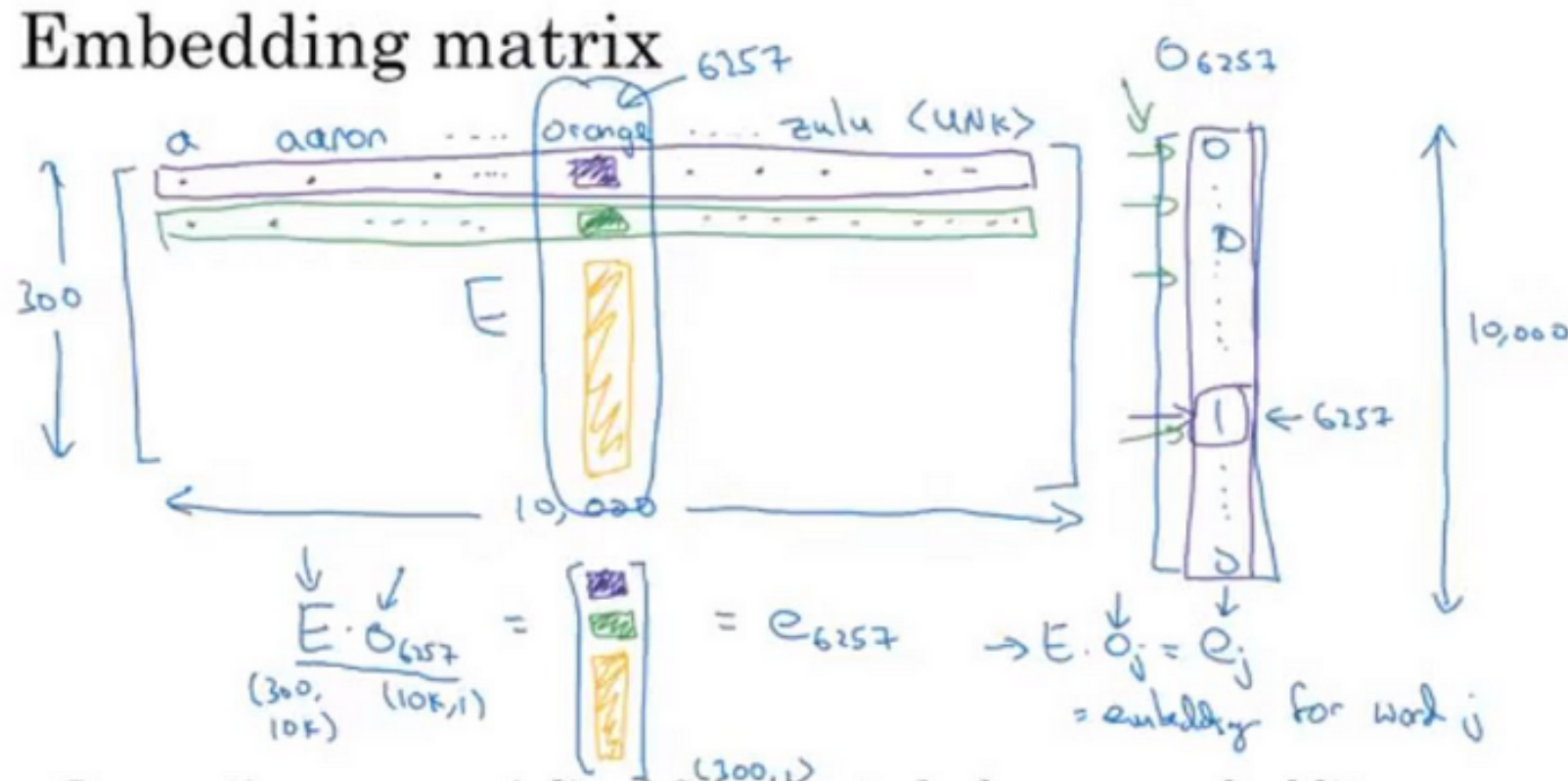
Now the model knows:

- ✓ Both "apple" and "orange" have **high food feature values**, so they're **similar**.
- ✓ "Man" and "Woman" differ in **gender**, not in food.

We store all 10,000 word vectors (each of size 300) in an **embedding matrix** E of shape $(10,000 \times 300)$. Each row corresponds to a word.

- For word "orange", index = 6257
- One-hot vector $o_{6257} = [0, \dots, 1 \text{ at } 6257, \dots, 0]$

$$E \times o_{6257} = e_{6257} \text{ (orange's embedding)}$$



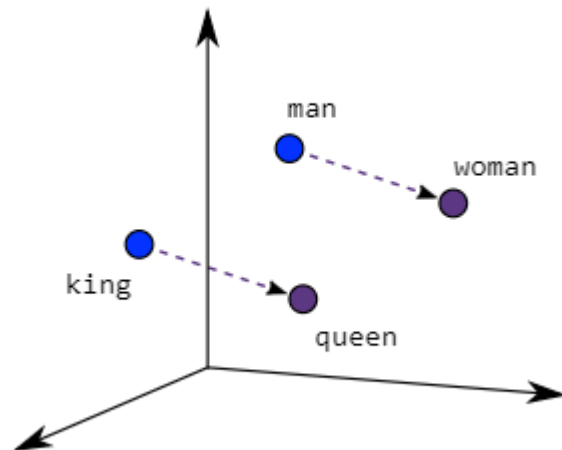
In practice, use specialized function to look up an embedding.

$E \cdot O_j = e_j$ (E =embedded matrix, O =1-hot representation)

 Aspect	 TF-IDF (Bag-of-Words)	 Word Embeddings (Dense Vectors)
Meaning/Semantics	No meaning captured. <i>“car” ≠ “vehicle”</i>	Captures meaning. <i>“car” ≈ “vehicle”</i>
Word Order	Ignored. <i>“Dog bites man” = “Man bites dog”</i>	Handled (in contextual embeddings). Understands the sentence meaning.
Context	Not captured. <i>“apple” in tech vs. fruit is treated the same</i>	Captures context. <i>“apple” (fruit) ≠ “Apple” (company)</i>
Vector Type	Sparse, high-dimensional (e.g., 10,000+ features)	Dense, low-dimensional (e.g., 300 features)
Generalization	Cannot handle unseen words (OOV = Out-of-Vocabulary)	Can estimate unseen words (esp. with subword models like fastText)
Pretraining	Not applicable — must train from scratch on your dataset	Pretrained on large corpora (e.g., GloVe, Word2Vec, BERT) — ready to use
Relationship Handling	No understanding of synonyms or analogies	Understands: <i>“king” - “man” + “woman” ≈ “queen”</i>
Topic Clustering	No clustering — words are isolated	<i>“apple”, “pear”, “banana”</i> cluster in the “fruit” region
Size & Efficiency	Inefficient — long and mostly zeros	Efficient — short, information-rich vectors
Use Case Fit	Good for simple, interpretable models	Better for deep learning and semantic tasks

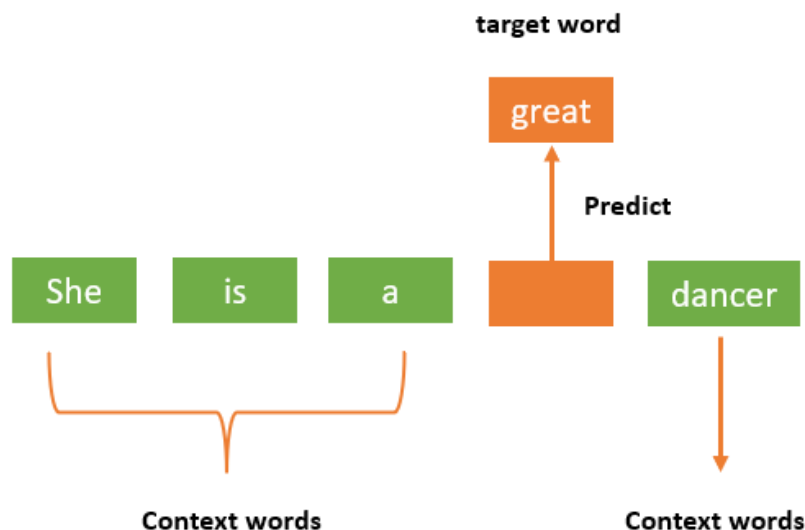
Word2Vec creates vectors of the words that are distributed numerical representations of word features – these word features could comprise of words that represent the context of the individual words present in our vocabulary. Word embeddings eventually help in establishing the association of a word with another similar meaning word through the created vectors.

As seen in the image below where word embeddings are plotted, similar meaning words are closer in space, indicating their semantic similarity.

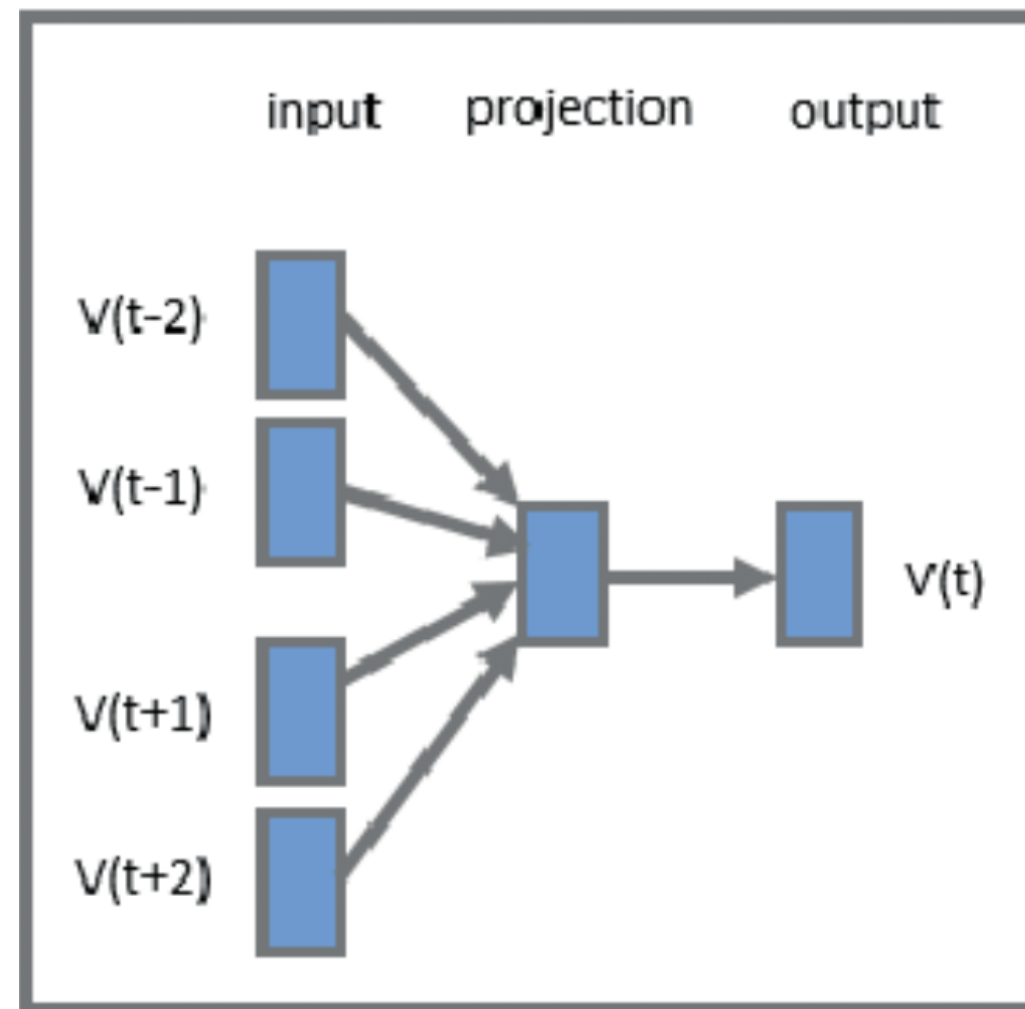


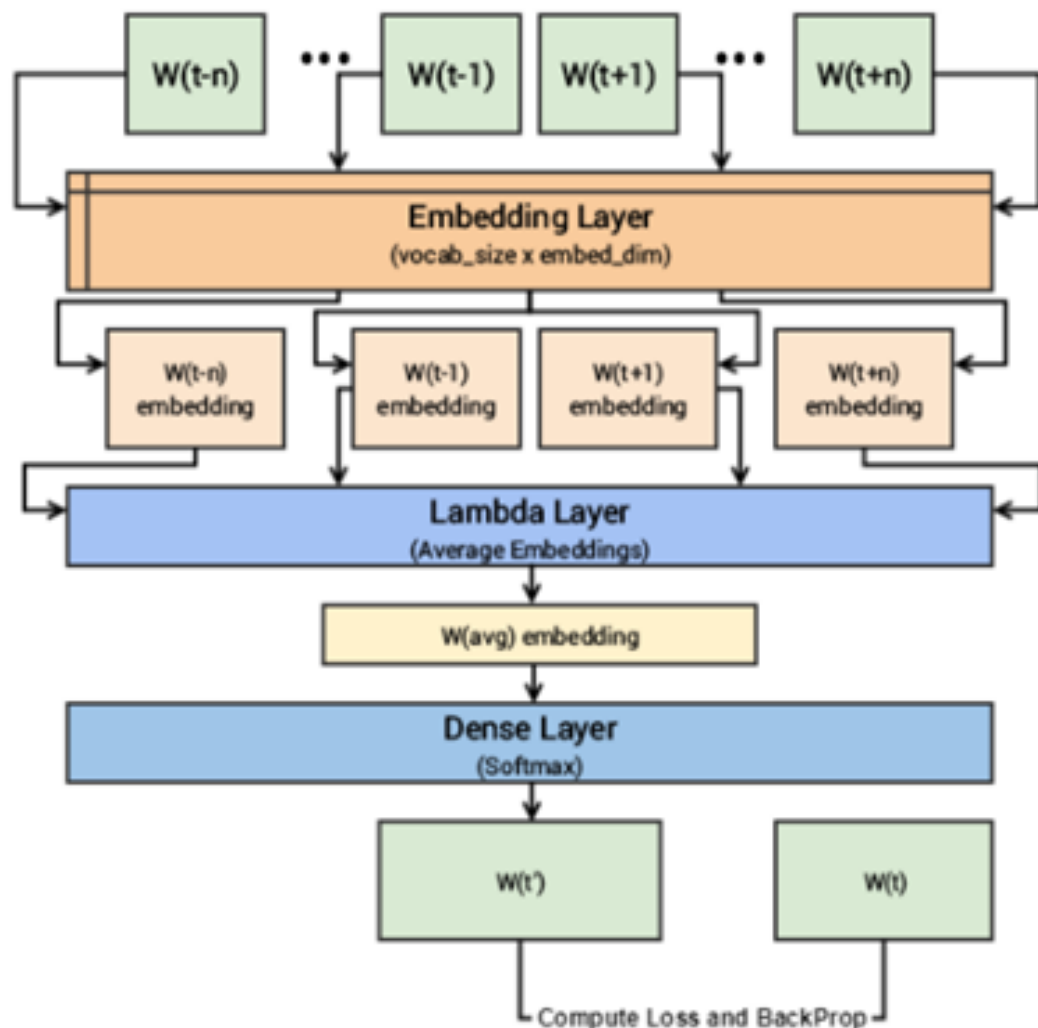
Two different model architectures that can be used by Word2Vec to create the word embeddings are the Continuous Bag of Words (CBOW) model & the Skip-Gram model.

- CBOW is a type of **word2vec** model introduced by Mikolov et al. (2013).
- The idea is to **predict a target word** based on the **surrounding context words**.
- Given a window of context words around a missing word, CBOW tries to predict the missing word.



For example, if we consider the sentence – “Word2Vec has a deep learning model working in the backend.”, there can be pairs of context words and target (center) words. If we consider a context window size of 1, we will have pairs like ([**deep**, **model**], **learning**), ([**model**, **in**], **working**), ([**a**, **learning**], **deep**) etc.



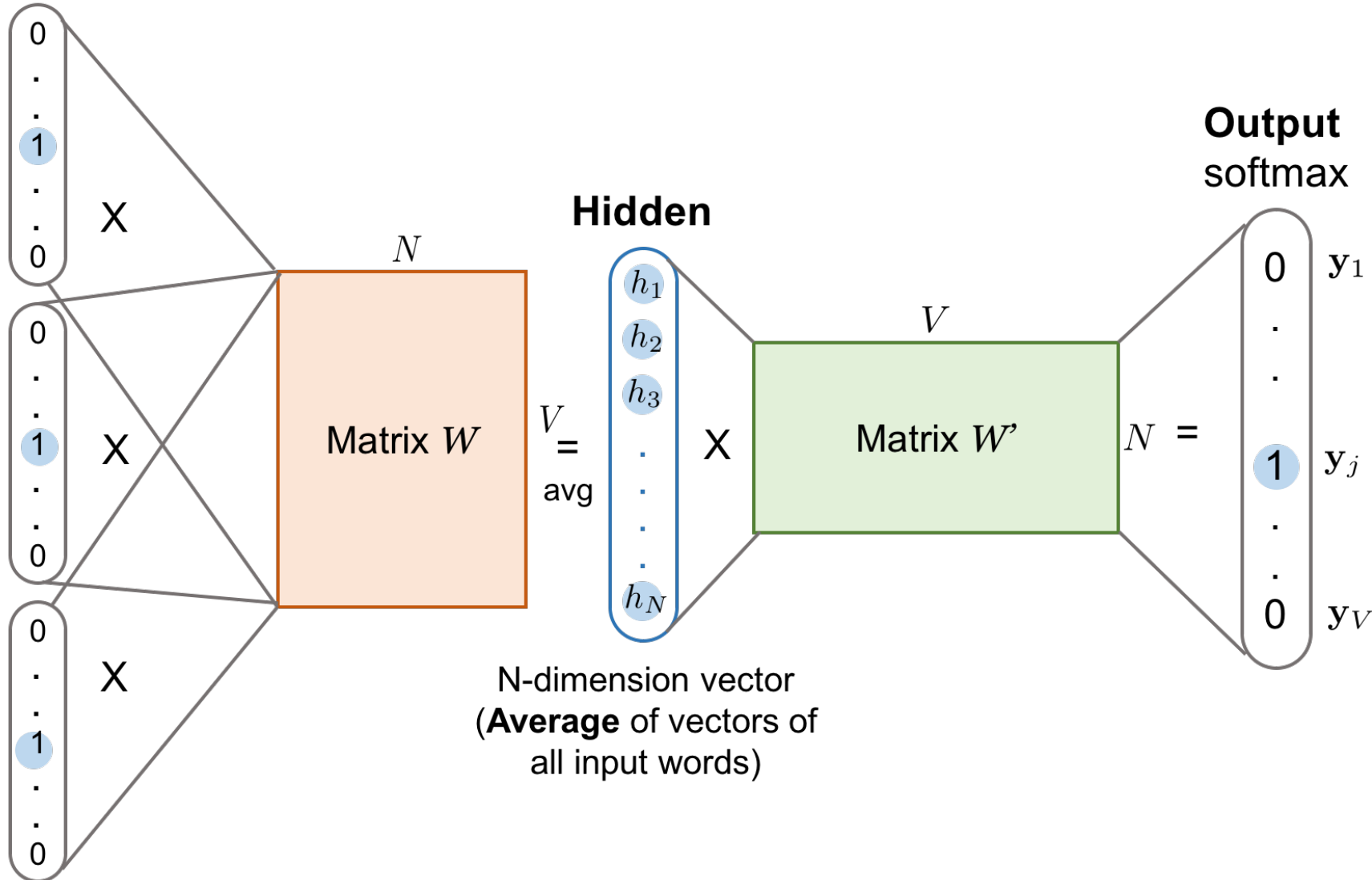


Visual depiction of the CBOW deep learning model

- The context words are first passed as an input to an embedding layer (initialized with some random weights) as shown in the Figure below.
- The word embeddings are then passed to a lambda layer where we average out the word embeddings.
- We then pass these embeddings to a dense SoftMax layer that predicts our target word. We match this with our target word and compute the loss and then we perform backpropagation with each epoch to update the embedding layer in the process.

We can extract out the embeddings of the needed words from our embedding layer, once the training is completed.

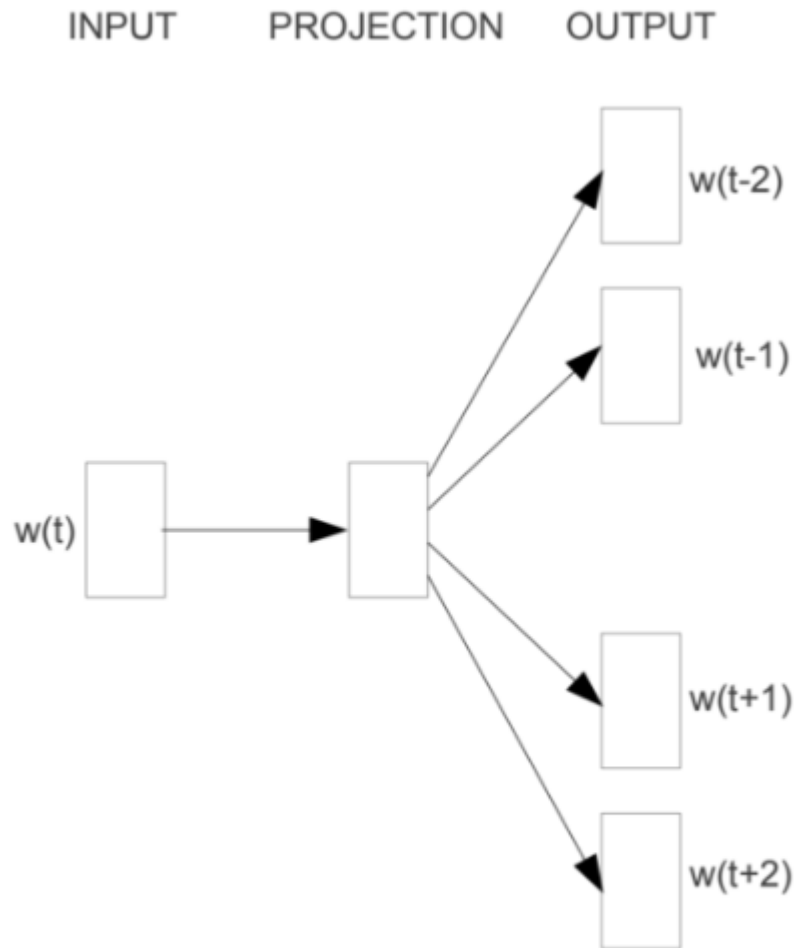
Input



The optimal window size depends on the nature of your data and the task at hand:

- **Smaller Window (e.g., 1–3):** Better for capturing syntactic relationships, such as grammar and word order.

- **Larger Window (e.g., 5–10):** More effective for capturing semantic relationships, such as word meanings and associations.



Skip-gram

The **Skip-Gram** model works by using a **target word** to predict its surrounding **context words** within a specified window size. The window size defines how many words before and after the target word are considered as context

For example, in the sentence:

"The quick brown fox jumps over the lazy dog"

- **Target Word:** "jumps"
- **Window Size:** 2

The context words would be

- **Before:** "quick", "brown"
- **After:** "over", "the"

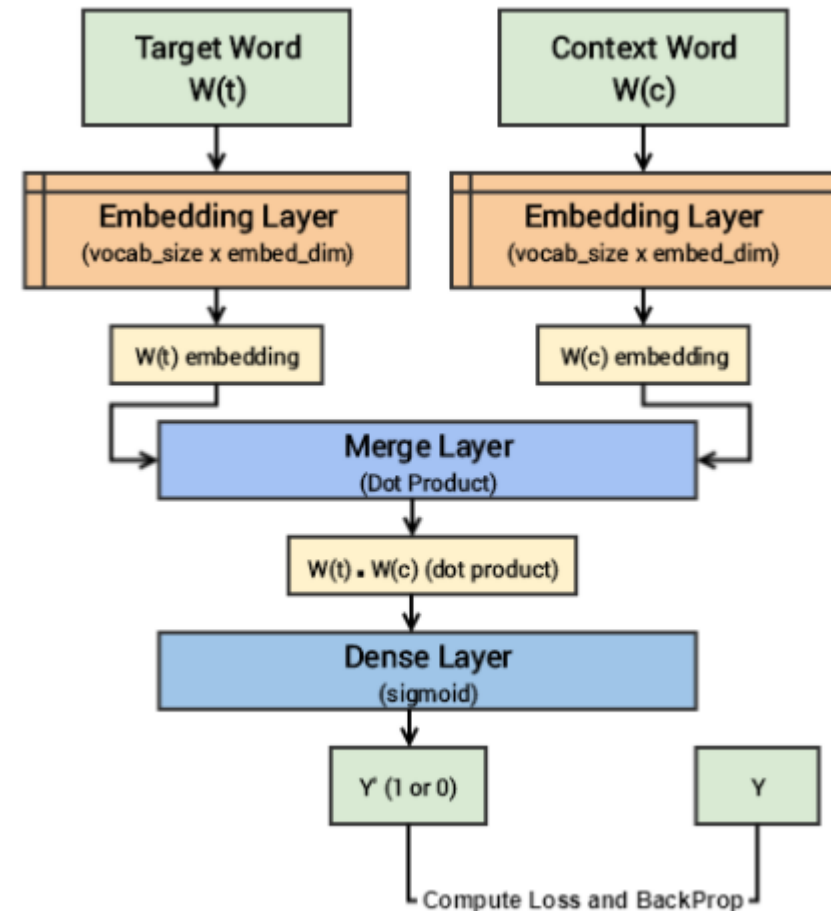
So, considering the same sentence – “Word2Vec has a deep learning model working in the backend.” and a context window size of 1, given the centre word ‘learning’, the model tries to predict [‘deep’, ‘model’] and so on.

Since the skip-gram model has to predict multiple words from a single given word, we feed the model pairs of (X, Y) where X is our input and Y is our label. This is done by creating **positive input samples and negative input samples**

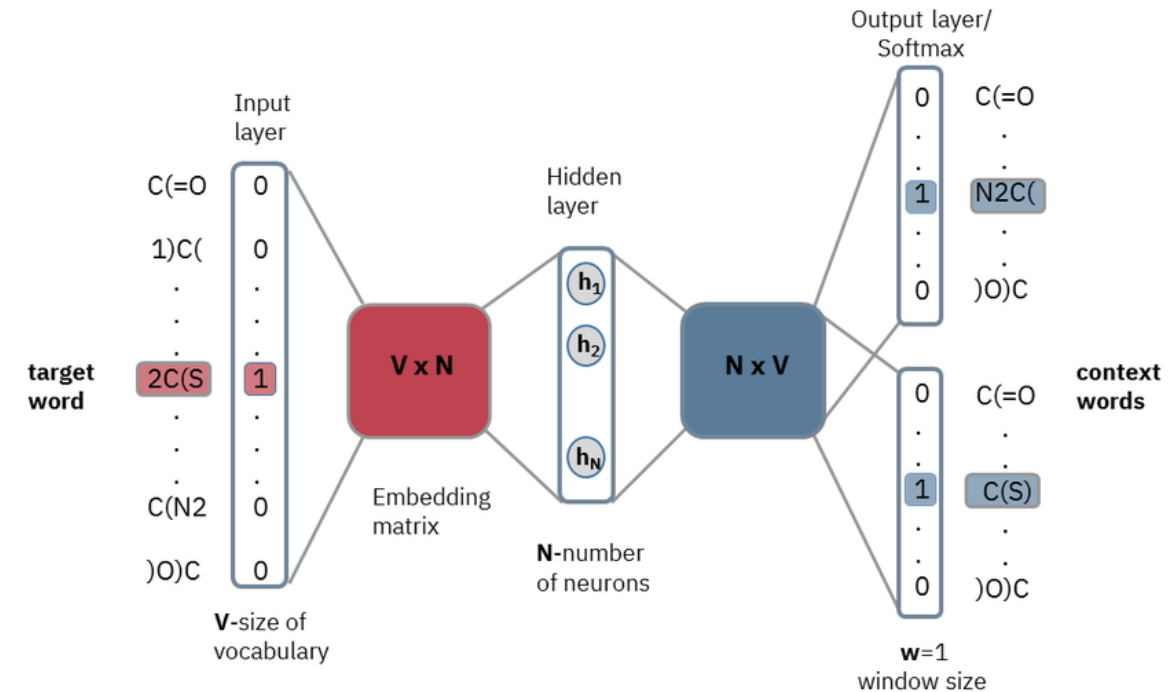
Positive Input Samples will have the training data in this form: [(target, context), 1] where the target is the target or centre word, context represents the surrounding context words, and label 1 indicates if it is a relevant pair.

Negative Input Samples will have the training data in the same form: [(target, random), 0]. In this case, instead of the actual surrounding words, randomly selected words are fed in along with the target words with a label of 0 indicating that it’s an irrelevant pair.

These samples make the model aware of the contextually relevant words and consequently generate similar embeddings for similar meaning words.

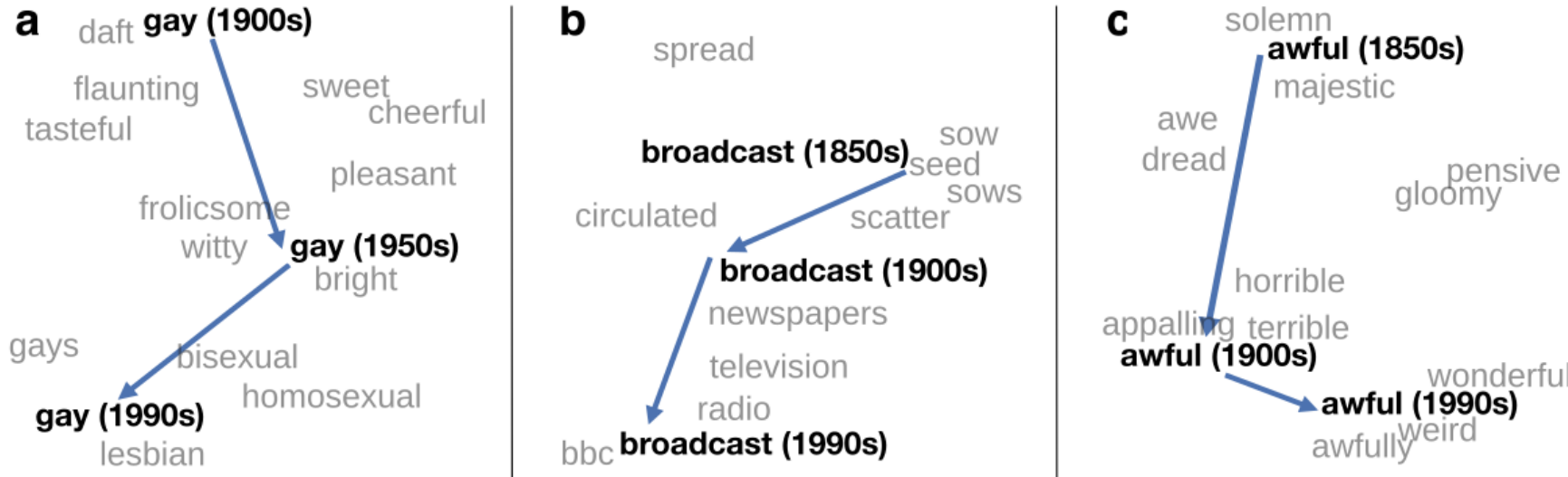


- Both the target and context word pairs are passed to individual embedding layers from which we get dense word embeddings for each of these two words.
- We then use a 'merge layer' to compute the dot product of these two embeddings and get the dot product value.
- This dot product value is then sent to a dense sigmoid layer that outputs either 0 or 1.
- The output is compared with the actual label and the loss is computed followed by backpropagation with each epoch to update the embedding layer in the process.



The major drawbacks of such Neural Network based Language Models are:

- High Training & Testing time
- Inability to capture the statistical information at a global level (they are not considering the count or co-occurrence of words in a global context).



GloVe is based on ratios of probabilities from the word-word co-occurrence matrix, combining the intuitions of count-based models while also capturing the linear structures used by methods like word2vec.

*“GloVe is a count-based, **unsupervised learning model** that uses **co-occurrence** (how frequently two words appear together) statistics at a **Global level** to model the vector representations of words.”*

Since the statistics are captured at a global level directly by the model, it is named as ‘Global Vectors’ model. GloVe is introduced by **Jeffrey Pennington, Richard Socher, Christopher D. Manning** of Stanford University in the year **2014** and has ever since gained popularity among NLP practitioners due to its simplicity and performance.

The basic idea behind the GloVe word embedding is to derive the relationship between the words from statistics. Unlike the occurrence matrix, the co-occurrence matrix tells you how often a particular word pair occurs together. Each value in the co-occurrence matrix represents a pair of words occurring together.

Given two corpus:

- 1.The fast cat wears no hat
- 2.The cat in the hat ran fast

Step 2: Define context window

Use a **window size = 1** (just the immediate neighbors) for simplicity.

Step 3: Build co-occurrence matrix XXX

- Rows and columns = vocabulary words in order.
- X_{ij} = how many times word j appeared *next to* word i within the context window.

Step 1: Extract unique words (vocabulary)

Let's lowercase all words and list unique tokens:

Vocabulary size = 8.	Index	Word
	0	the
	1	fast
	2	cat
	3	wears
	4	no
	5	hat
	6	in
	7	ran

Step 4: Count co-occurrences in each sentence

Sentence 1: "the fast cat wears no hat"

Position	Word	Context words (window=1)
0	the	fast
1	fast	the, cat
2	cat	fast, wears
3	wears	cat, no
4	no	wears, hat
5	hat	no

From this sentence, count pairs:

- (the, fast) $\rightarrow X_{the,fast} += 1$
- (fast, the), (fast, cat) $\rightarrow X_{fast,the} += 1, X_{fast,cat} += 1$
- (cat, fast), (cat, wears) $\rightarrow X_{cat,fast} += 1, X_{cat,wears} += 1$
- (wears, cat), (wears, no) $\rightarrow X_{wears,cat} += 1, X_{wears,no} += 1$
- (no, wears), (no, hat) $\rightarrow X_{no,wears} += 1, X_{no,hats} += 1$
- (hat, no) $\rightarrow X_{hat,no} += 1$

Sentence 2: "the cat in the hat ran fast"

Position	Word	Context words (window=1)
0	the	cat
1	cat	the, in
2	in	cat, the
3	the	in, hat
4	hat	the, ran
5	ran	hat, fast
6	fast	ran

Count pairs:

- (the, cat) $\rightarrow X_{\text{the,cat}} + = 1$
- (cat, the), (cat, in) $\rightarrow X_{\text{cat,the}} + = 1, X_{\text{cat,in}} + = 1$
- (in, cat), (in, the) $\rightarrow X_{\text{in,cat}} + = 1, X_{\text{in,the}} + = 1$
- (the, in), (the, hat) $\rightarrow X_{\text{the,in}} + = 1, X_{\text{the,hate}} + = 1$
- (hat, the), (hat, ran) $\rightarrow X_{\text{hat,the}} + = 1, X_{\text{hat,ran}} + = 1$
- (ran, hat), (ran, fast) $\rightarrow X_{\text{ran,hat}} + = 1, X_{\text{ran,fast}} + = 1$
- (fast, ran) $\rightarrow X_{\text{fast,ran}} + = 1$

	the	fast	cat	wears	no	hat	in	ran
the	0	1	1	0	0	1	1	0
fast	1	0	1	0	0	0	0	1
cat	1	1	0	1	0	0	1	0
wears	0	0	1	0	1	0	0	0
no	0	0	0	1	0	1	0	0
hat	1	0	0	0	1	0	0	1
in	1	0	1	0	0	0	0	0
ran	0	1	0	0	0	1	0	0

- The matrix is **not symmetric** because co-occurrence depends on the order in the sentence (word vs context word).
- You could make it symmetric by summing $X_{ij} + X_{ji}$, but GloVe handles the asymmetry.

$$J = \sum_{i,j} f(X_{ij}) (v_i \cdot u_j - \log(X_{ij}))^2$$

- v_i and u_j are the word vectors (embeddings) for word i and context word j .
- X_{ij} is the co-occurrence count between word i and j .
- $f(X_{ij})$ is a weighting function (usually designed to reduce the impact of very frequent or very rare co-occurrences).

Step 6: Initialize embeddings

- Randomly initialize vectors $v_i, u_j \in \mathbb{R}^d$ for all words in vocab.
- Initialize bias terms b_i, c_j for words and contexts (sometimes included in the model).

Example:

- Dimension $d = 50$ (or any small number).
- $v_i, u_j \sim \text{random values}$

Step 7: Define the weighting function $f(x)$

The original GloVe paper suggests:

$$f(x) = \begin{cases} (x/x_{max})^\alpha & \text{if } x < x_{max} \\ 1 & \text{otherwise} \end{cases}$$

where $x_{max} = 100, \alpha = 0.75$.

In our tiny example, you could simplify it as:

- $f(x) = 1$ if $x > 0$,
- else $f(x) = 0$ (skip zero counts).

Step 8: Optimize embeddings

Minimize J using **gradient descent** or other optimizers like Adam:

- For each pair (i, j) where $X_{ij} > 0$:
 - Calculate prediction error: $e_{ij} = v_i \cdot u_j - \log(X_{ij})$
 - Calculate gradient with respect to v_i, u_j
 - Update v_i, u_j (and biases if used) to reduce error.

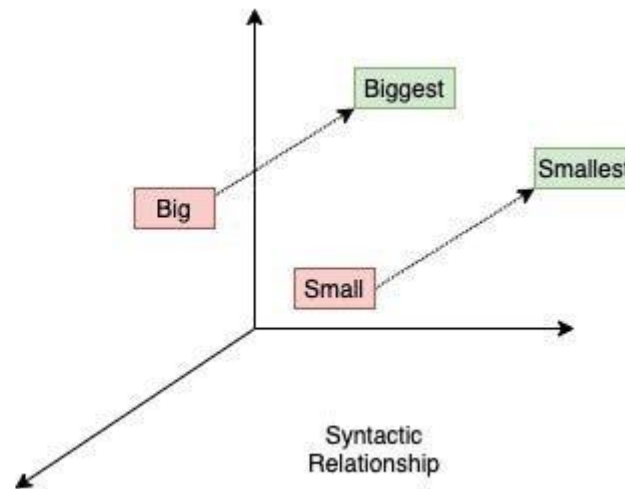
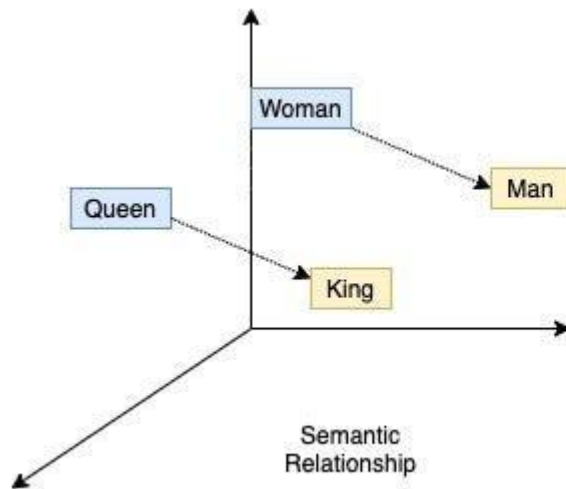
Repeat over multiple epochs until convergence or for fixed iterations.

Step 9: Combine embeddings

Final embedding for word i is often:

$$w_i = v_i + u_i$$

or just use v_i .



Queen-Woman = King-Man
 Rewriting,
Queen-Woman+Man = King

Relationship Type	Examples	Word Vector Behavior
Semantic	man → woman, king → queen	Word vectors form consistent differences along meaning axes (e.g., gender, royalty)
Syntactic	big → biggest, run → running	Vectors show consistent changes in form (e.g., tense, comparison, pluralization)

You can **test** this by checking vector arithmetic:

- **Semantic example:**

$$\text{vec}(\text{King}) - \text{vec}(\text{Man}) + \text{vec}(\text{Woman}) \approx \text{Queen}$$

- **Syntactic example:**

$$\text{vec}(\text{Big}) - \text{vec}(\text{Small}) + \text{vec}(\text{Smallest}) \approx \text{Biggest}$$

Synsets (short for "**synonym sets**") are a fundamental concept in **WordNet**, a large lexical database of English developed at Princeton.

A **synset** is a group of words (or phrases) that are **synonyms**, meaning they express the **same concept or meaning**.

Each synset is also associated with:

- A **definition** (gloss)
- One or more **example sentences**
- Semantic relationships** (like hypernyms, hyponyms, antonyms)

For the concept of **car**, WordNet provides this **synset**:

car.n.01

- Synonyms: car, auto, automobile, machine, motorcar
- Definition: a motor vehicle with four wheels; usually propelled by an internal combustion engine
- Example: "He needs a car to get to work."

WordNet organizes words into **four major parts of speech**:

- n = noun
- v = verb
- a = adjective
- r = adverb

Each synset is assigned a unique **ID**, like bank.n.01, run.v.05, etc.

upGrad



Thank You!!